

AI Studio API Call Safety & Concurrency Mitigation

LG Magazine (mybook.lgacademy.com) — LLM request resilience under event load
Prepared 2026-06-24 · Build main `a7e00c0` · Provider: LG AI Studio (aistudio.singlex.com)

1. Executive Summary

The magazine's personalized content is generated by LLM calls routed through **LG AI Studio**. The single hard constraint is **concurrency**: the AI Studio account returns generic HTTP 500 errors once roughly **~20 simultaneous in-flight calls** are exceeded. This is an *account / gateway* limit — it is **not** solved by the 10-key rotation (which only spreads token quota) and is **not** changed by which model version is used.

Token quota is NOT the bottleneck. LG's stated per-minute limits are very large (Claude Sonnet 4.6 = 6,000,000 tokens/min). At our concurrency ceiling we use only a small fraction of that. The problem to mitigate is **concurrent calls**, not tokens.

Mitigation is in two parts: **(A)** an application-level safety stack that is already shipped in the current build, and **(B)** operational actions — most importantly asking LG to raise the concurrency limit, since that is the only lever that increases real capacity.

2. The Constraints (per LG CNS, 2026-06-24)

Model (per CSP)	Requests / min	Tokens / min
GPT 5.4 nano	9,941	9,940,000
Claude Sonnet 4.6	— (not stated)	6,000,000
Gemini 2.5 flash	4,024	— (not stated)

These are **per-minute aggregate** limits. They are generous and not our constraint. LG did **not** state a **concurrency** limit — and concurrency (~20 observed) is what actually causes failures under load.

~20

Observed concurrent-call ceiling (account-level). Beyond this → HTTP 500.

6M / min

Sonnet token budget — ample headroom; never the limiting factor at our scale.

3. Mitigation — Layered Defense (already in the build)

Layer	What it does	Protects against
Concurrency gate	Caps concurrent upstream calls per replica (<code>AISTUDIO_MAX_CONCURRENCY=5</code>); excess queues.	The ~20-concurrent 500 wall.
Retry + backoff	Transient 5xx/429 retried twice with exponential backoff, inside the gate slot (adds no concurrency).	Brief upstream blips.
10-key rotation + cooldown	On a quota hit, the code is parked (60s per-minute / 30min daily) and the next code is used.	Per-code token quota exhaustion.
Async job queue <code>opt-in</code>	<code>LLM_ASYNC=1</code> : POST returns a jobId; client polls. No held-open connections.	Timeouts & 504s under a rush.
Auto re-auth	Expired JWT refreshes and the call retries once.	Token expiry mid-traffic.
Stub fallback	On any final failure, returns template content so the magazine never hard-fails.	Total user-facing failure.

4. Configuration Rules (must hold in production)

1. **Keep concurrency under the account ceiling:** `AISTUDIO_MAX_CONCURRENCY × replicas < limit` . Current: `5 × 3 = 15 < ~20` ✓. Adding API codes does **not** raise this (the wall is per-account, not per-code).

2. **Async needs sticky sessions.** The job queue is in-memory per replica, so enable `LLM_ASYNC=1` together with the nginx upstream change `hash $cookie_qrius_session consistent;` — otherwise a poll can land on a replica that doesn't hold the job (404).
3. **Timeouts must align.** In sync mode keep `AISTUDIO_QUEUE_TIMEOUT_MS` (20s) under nginx `proxy_read_timeout` (≥ 60 s). In async mode requests return instantly, so this is moot.

5. Action Plan

#	Action	Owner	Why it matters
1	Ask LG for the concurrency limit (동시 호출 한도) and whether it can be raised to the target.	Us → LG CNS	The only lever that adds real capacity. Everything else just degrades gracefully.
2	Enable async: <code>LLM_ASYNC=1</code> + nginx sticky hash, then redeploy.	Us (server)	Users wait in line instead of timing out under a rush.
3	Confirm the model bound to the <code>LG_B00K_*</code> codes (dashboard / probe).	Us / LG	Model is set on AI Studio's side, not in our code.
4	Load-test with async on (k6, off-peak): verify low 429s, no stub flood, no restarts.	Us	Confirms the stack holds at target load before the event.
5	Monitor during the event: watch for <code>fell back to stub</code> / <code>exhausted</code> / 429 bursts.	Us	Early signal that the ceiling is being hit.

6. On "switch to Sonnet 4.6 to handle 100+ concurrent"

Model version $\neq$ concurrency capacity. Switching 4.5 → 4.6 will not, by itself, unlock 100 concurrent — the ~20 ceiling is an account property, not a model property. It *may* help only if LG confirms Sonnet 4.6 carries a higher concurrency allowance (they have not stated one).

- The model swap is a **dashboard rebinding** (LG / workspace owner), not a code change; our only lever is which codes/prompt index we point to.
- To reach 100 concurrent you need LG to **raise the account concurrency limit**. A concurrency-sweep test (10→20→40→60→100) can *measure* today's ceiling but cannot raise it, costs tokens, and should be coordinated with LG and run off-peak on a test code.